University of Toronto at Scarborough
CSCC37—Numerical Algorithms for Computational Mathematics, Fall 2024

Assignment #2: GE, LU ftzn / Pivoting / Complexity

Due: October 27, 2024 at 11:45 p.m.
This assignment is worth 10% of your final grade.

Warning: Your electronic submission on *MarkUs* affirms that this assignment is your own work and no one else's, and is in accordance with the University of Toronto Code of Behaviour on Academic Matters, the Code of Student Conduct, and the guidelines for avoiding plagiarism in CSCC37.

This assignment is due by 11:45 p.m. October 27. If you haven't finished by then, you may hand in your assignment late with a penalty as specified in the course information sheet.

[15] 1. In lecture we showed how Gauss transforms could be used to compute the $A = LU$ or $PA = LU$ factorizations of $A \in \mathbb{R}^{n\times n}$. When doing examples we assumed certain properties of these transforms in order to simplify operations such as matrix inversion. In this question, you will attempt to prove these properties.

(a) Let $\mathcal{L}_k$ be the Gauss transform of order n used to annihilate elements in the k-th column during the k-th stage of Gaussian Elimination. Show that $\mathcal{L}_k$ can be written compactly in matrix-vector notation as

$$\mathcal{L}_k = I - m_k e_k^T, \tag{1}$$

where e_k is the k-th column of the identity matrix and

$$e_i^T m_k = 0, \quad i = 1, 2, \ldots, k. \tag{2}$$

(b) Using the notation in (a), prove that $\mathcal{L}_k^{-1}$ can be computed simply by changing the sign of each multiplier (i.e. by changing the sign of each non-zero element in m_k).

(c) Using (a) and (b), prove that $(\mathcal{L}_k\mathcal{L}_j)^{-1}$, $j < k$, can be computed simply by adding the two transforms, changing the sign of each multiplier, and subtracting the identity.

(d) Let $\mathcal{P}_i \equiv \mathcal{P}_{ij} \in \mathbb{R}^{n\times n}$ be the permutation matrix which, when pre-multiplied to another matrix, swaps rows i and j of that matrix with $i < j$. Using the notation for $\mathcal{L}_k$ in (a), prove that $\tilde{\mathcal{L}}_k = \mathcal{P}_i\mathcal{L}_k\mathcal{P}_i$ is just $\mathcal{L}_k$ with multipliers i and j swapped, but only if $i > k$.

[15] 2. Consider the linear system $Ax = b$ where

$$A = \begin{bmatrix} 2 & 5 & 10 \\ 8 & 32 & 8 \\ 1 & 8 & 13 \end{bmatrix}, \quad b = \begin{bmatrix} 7 \\ -16 \\ 6 \end{bmatrix}.$$

(a) Compute the $PA = LU$ factorization of A. Use exact arithmetic. Show all intermediate calculations, including Gauss transforms and permutation matrices.

(b) Use the factorization computed in (a) to solve the system.

(c) Why is Gaussian Elimination usually implemented as in this question (i.e., $PA = LU$ is computed separately, and then the factorization is used to solve $Ax = b$)?

[20] 3. In lecture we saw that Gaussian Elimination with row partial pivoting leads to a more stable factorization of $A \in \mathbb{R}^{n \times n}$. A stable factorization can be *provably guaranteed* if we use *full* pivoting, which employs both row and column interchanges before the k-th stage of the elimination to ensure that the largest element in magnitude in the entire $(n-k) \times (n-k)$ lower-right submatrix finds its way to the pivot position.

(a) Sketch the general structure of an $n \times n$ matrix just before the k-th stage of the elimination, and show how row and column interchanges can move the largest element in magnitude in the current matrix to the pivot position. Remember that previously annihilated elements from the first $k-1$ stages **must not** be reinstated (filled in) by this move.

(b) Full pivoting leads to a $PAQ = LU$ factorization, where P and Q are permutation matrices. Show algebraically how this factorization can be used to solve $Ax = b$ with forward elimination and backsolve on a lower and upper triangular system, respectively. We know that P effectively reorders the equations in the original system. What is the effect of Q on the original system?

(c) Repeat #2, but this time use *full* pivoting.

[10] 4. Let $A \in \mathbb{R}^{n \times n}$ be a nonsingular matrix, and let $x_i, t_i \in \mathbb{R}^n$, $i = 1, \ldots, k$.

(a) Explain how Gaussian Elimination with partial pivoting can be used to solve the k linear systems

$$Ax_1 = t_1,\ Ax_2 = t_2,\ \ldots,\ Ax_k = t_k$$

as *efficiently as possible*. Derive the complexity of your approach by counting *flops* (multiplication/addition pairs).

Note: Your final operation count will include both n and k.

(b) For $k = n$ and a suitable choice for the n right-hand-sides $t_1, t_2, \ldots, t_n$, your algorithm in (**a**) can be used to compute A^{-1}. Explain how this can be done.

(c) A possible scheme for solving $Ax = b$ is to first compute A^{-1} using (**b**), and then compute the matrix-vector product $x = A^{-1}b$. Is this scheme preferable to the usual Gaussian Elimination algorithm with $PA = LU$ factorization, forward elimination, and backsolve? Give operation counts to justify your answer.

(d) Instead of using (**b**) to compute A^{-1}, we could use Cramer's rule. Would matrix inversion with Cramer's rule followed by the matrix-vector product $x = A^{-1}b$ be preferable to the usual Gaussian Elimination algorithm? Give operation counts to justify your answer.

[total: 60 marks]